MachineLearningHW4_WushuangRui

December 12, 2025

1 Part A — Backpropagation (Hand Calculation)

Given:

x1 = 1

x2 = 2

w1 = 0.10

w2 = -0.20

b = 0.00

target y = 1.0

learning rate $\eta = 0.1$

1.1 1) Forward Pass

$$z = w1 * x1 + w2 * x2 + b = (0.10) * 1 + (-0.20) * 2 + 0.00 = 0.10 - 0.40 + 0.00 = -0.30$$

$$\hat{y} = z = -0.30$$

1.2 2) Loss

$$L = (1/2) * (\hat{y} - y)^2 = 0.5 * (-0.30 - 1.0)^2 = 0.5 * (-1.30)^2 = 0.5 * 1.69 = 0.845$$

1.3 3) Backpropagation

We have L = (1/2)(ŷ - y)^2, so:

$$\partial L/\partial \hat{y} = (\hat{y} - y) = -0.30 - 1.0 = -1.30$$

Since ŷ = z, we have ∂ ŷ/ ∂ z = 1, therefore:

$$\partial L/\partial z = \partial L/\partial \hat{y} * \partial \hat{y}/\partial z = (-1.30) * 1 = -1.30$$

z = w1*x1* + *w2*x2 + b, so:

$$\partial z/\partial w1 = x1 = 1, \partial z/\partial w2 = x2 = 2, \partial z/\partial b = 1$$

Thus:

$$\partial L/\partial w1 = \partial L/\partial z * \partial z/\partial w1 = (-1.30) * 1 = -1.30$$
$$\partial L/\partial w2 = \partial L/\partial z * \partial z/\partial w2 = (-1.30) * 2 = -2.60$$
$$\partial L/\partial b = \partial L/\partial z * \partial z/\partial b = (-1.30) * 1 = -1.30$$

1.4 4) Gradient Descent Updates

Update rule: parameter_new = parameter - η * (gradient)

$$w1_new = w1 - \eta * (\partial L/\partial w1) = 0.10 - 0.1 * (-1.30) = 0.10 + 0.13 = 0.23$$
$$w2_new = w2 - \eta * (\partial L/\partial w2) = -0.20 - 0.1 * (-2.60) = -0.20 + 0.26 = 0.06$$
$$b_new = b - \eta * (\partial L/\partial b) = 0.00 - 0.1 * (-1.30) = 0.00 + 0.13 = 0.13$$

2 Part B — Colab Notebook

2.1 A.1 Search the Neural Net code for the comment that says "where the magic happens". This is the beginning of the core loop in PyTorch, where the learning of machine learning happens. There are 5 lines following the comment that indicate the 5 steps. In your own words, summarize what those 5 steps are and how they induce what we call learning.

The five steps in the training loop are:

1. `optimizer.zero_grad()` Clears gradients from the previous batch so they do not accumulate.
2. `logits = model(x)` Forward pass: the model produces predictions from the input data.
3. `loss = criterion(logits, y)` Computes the loss by comparing predictions with ground-truth labels using CrossEntropyLoss.
4. `loss.backward()` Backpropagation: computes gradients of the loss with respect to all model parameters.
5. `optimizer.step()` Updates the weights of the model using the computed gradients.

Learning occurs because these steps are repeated for each batch, continuously adjusting the model parameters to reduce the loss.

2.2 A.2 What is a residual connection in a neural net? If you search the example code, you'll find the definition. Why would you want a residual connection between layers?

A **residual connection** adds the input of a layer directly to its output (i.e., `out + x`), as implemented in the `ResidualBlock`.

Why it is useful:

- **Makes deep networks easier to optimize** - The `out + x` operation provides a shortcut for "identity mapping" to the network. If some layers fail to learn useful transformations, the model can at least degrade to something close to an identity transformation, preventing performance from worsening with each added layer.

- **Improves gradient flow and reduces vanishing gradient problems** - The gradient can be backpropagated more directly to the earlier layers along the skip path, resulting in more stable training and faster convergence (especially noticeable with a larger number of layers).
- **Allows layers to learn small refinements instead of full transformations** - The module only needs to learn to add a "residual" to x, which is usually simpler than directly learning the complete mapping.

2.3 A.3 In the plant classifier, the label had to be converted from a string to an integer to a tensor. Why?

Tensor is a multi-dimensional numerical array that represents data in neural networks and enables efficient computation and automatic differentiation.

- Neural networks and loss functions operate only on numeric data.
- `CrossEntropyLoss` requires labels as **integer class indices**, not strings.
- Labels must be **PyTorch tensors** so they can be moved to the GPU and used in backpropagation.

2.4 A.4 For the RNN vs 1D CNN example: how is a 1D CNN comparable to an RNN (what is the observation type they both address)? When is one better than the other?

2.4.1 How do 1D CNNs compare to RNNs (in terms of the types of observations they handle)?

Both are used for one-dimensional sequences: for example, time series, audio waveforms, and sensor sequences. The observations here are "multidimensional feature sequences arranged in time."

1D CNNs use convolutional kernels that slide along the time dimension to learn local temporal patterns (short-term dependencies); RNNs/LSTMs model temporal dependencies through recurrent state updates (capable of expressing longer-range dependency structures).

2.4.2 When are 1D CNNs better?

- When the key patterns are local and translation-invariant (a certain short-term shape/fluctuation is important regardless of when it occurs).
- They are usually faster to train, have better parallelism (convolution is GPU-friendly), are simpler to implement, and have fewer gradient problems.
- CNNs are often more efficient than RNNs when the sequences are very long.

2.4.3 When are RNNs/LSTMs better?

- When the task requires explicitly utilizing sequential state updates, or is more sensitive to "sequence/context."
- When it's necessary to capture more complex temporal dynamics (e.g., some dependencies that cannot be solved by local filtering).

Although LSTMs can mitigate vanishing gradients, they are harder to parallelize and slower to train compared to CNNs.